

GRUPPO: CRITICAL SECTION

MAIL DEL REFERENTE DEL GRUPPO: antonella.candia@studio.unibo.it

MEMBRI:

- ANTONELLA BELEN, CANDIA, Matr: 0001097853
- AURORA, CORTECCHIA, Matr: 0001069738
- ALESSIA, DIETTI, Matr: 0001097971
- SABRINA, XU, Matr: 0001070429

Indice:

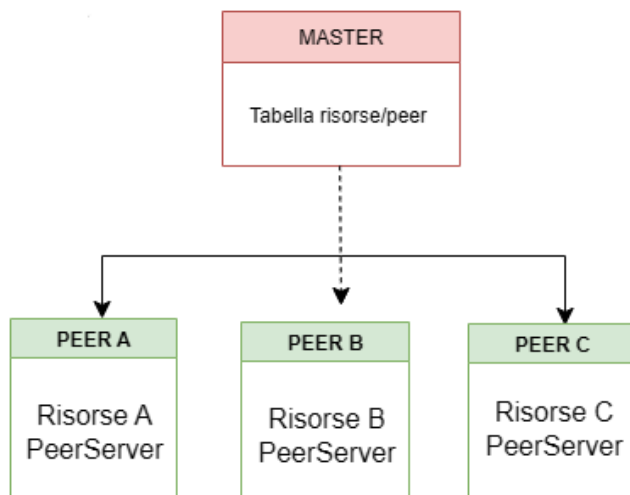
- 1. Descrizione del progetto**
- 2. Componenti del progetto (.java)**
- 3. Meccanismi di concorrenza e sincronizzazione**
- 4. Funzionalità e utilizzo**
- 5. Compilazione ed esecuzione**
- 6. Conclusione e riflessioni finali**
- 7. Suddivisione del lavoro nel gruppo**

1. Descrizione del progetto

Il progetto è stato sviluppato nell'ambito del corso **Laboratorio di Sistemi Operativi (LABSO) – A.A. 2024/2025**, con l'obiettivo di realizzare un **sistema distribuito per la condivisione di file** basato sul modello **client-server** con funzionalità **Peer-to-Peer (P2P)**.

L'architettura si fonda su:

- **un nodo Master centrale**, che funge da punto di coordinamento e mantiene la tabella globale delle risorse disponibili nella rete;
- **una rete di Peer**, che interagiscono con il Master per la registrazione e la ricerca delle risorse, ma si scambiano i file direttamente in maniera distribuita tramite un piccolo server interno dedicato.



Questo approccio ibrido combina i vantaggi della centralizzazione (coordinamento e visione globale delle risorse) con quelli della decentralizzazione (scalabilità e riduzione del carico sul Master).

Dal punto di vista tecnico, il sistema è stato implementato in Java 17, utilizzando:

- socket TCP per la comunicazione tra nodi;
- thread per la gestione concorrente delle connessioni;
- meccanismi di sincronizzazione (ReentrantLock e strutture thread-safe) per garantire l'integrità delle strutture dati condivise.

Sono state inoltre introdotte funzionalità aggiuntive per aumentare la robustezza, come:

- un file di configurazione config.properties, che consente di parametrizzare host, porte e timeout senza modificare il codice sorgente;
- un meccanismo di round robin per i download: in caso di fallimento con un Peer, il Master fornisce un nuovo fornitore finché la lista non si esaurisce.

La struttura generale del progetto può essere sintetizzata in due livelli principali:

- **Master**: mantiene la tabella globale delle risorse (mappando ogni file ai Peer che lo possiedono), coordina le operazioni di lookup, aggiorna lo stato della rete e tiene un log delle operazioni di download. Non trasmette file.
- **Peer**: consente agli utenti di aggiungere risorse, pubblicarle al Master, ricercarle nella rete e scaricarle da altri Peer tramite connessioni dirette.

In sintesi, il progetto realizza un sistema distribuito capace di gestire registrazione, ricerca e download di risorse con un modello cooperativo Master-Peer, mantenendo un buon bilanciamento tra semplicità e realismo.

1.1 Architettura generale del sistema

Il sistema realizzato si basa su un'architettura distribuita di tipo client-server con estensioni Peer to Peer, nella quale il nodo Master svolge il ruolo di coordinatore centrale e i Peer rappresentano le entità distribuite che interagiscono sia con il Master sia tra loro.

- **Master** (livello di Controllo): Il Master è il punto di riferimento del sistema. Mantiene una tabella aggiornata che associa le risorse disponibili e la lista dei Peer che le possiedono. Ogni Peer all'avvio, deve connettersi al Master per registrarsi, comunicando la propria porta di ascolto e l'elenco iniziale delle risorse che possiede. Il Master registra i Peer al momento della connessione e fornisce loro informazioni necessarie per localizzare le risorse. Non memorizza alcuna risorsa e non partecipa direttamente al trasferimento dei dati, ma svolge funzioni di coordinamento e supervisione.

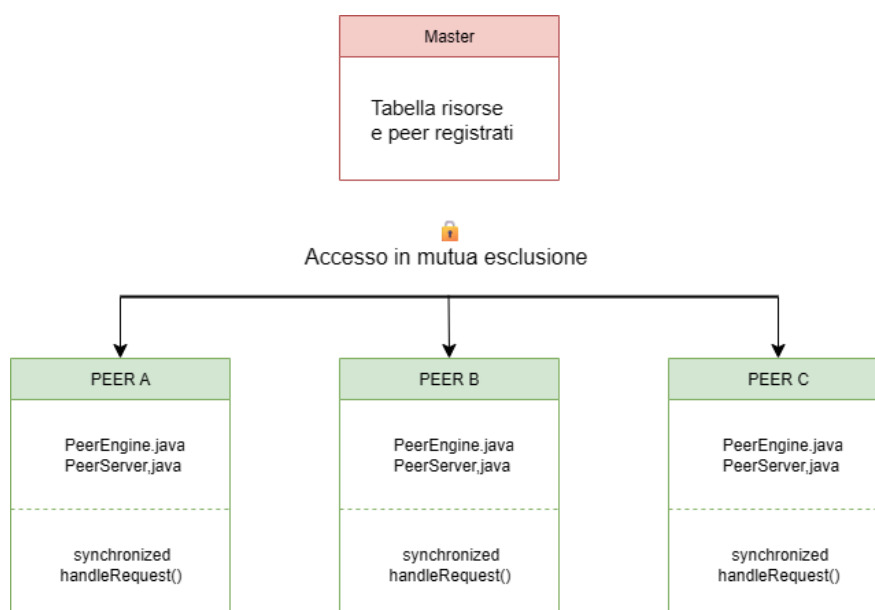
Le funzioni principali del Master includono:

- mantenere la tabella delle risorse disponibili;
- fornire la lista dei Peer che possiedono una risorsa;
- aggiornare la tabella delle risorse dinamicamente e mettere in attesa le altre richieste durante gli aggiornamenti;
- mantenere un log delle operazioni di download.

- **Peer**: sono i nodi attivi che offrono e richiedono risorse. Ogni Peer comunica con il Master per registrarsi e per richiedere la disponibilità di un file, ma lo scambio effettivo delle risorse avviene direttamente tra Peer, tramite un server interno dedicato.

Le funzioni principali dei Peer includono:

- registrarsi al Master con l'elenco delle risorse locali;
- come Client: chiede al Master dove scaricare una risorsa;
- come Server: tramite un server interno dedicato consente ad altri Peer di scaricare risorse;
- gestione concorrente: il server interno gestisce più richieste simultaneamente, assegnando un thread dedicato per ogni connessione in entrata;
- notificare il Master quando aggiunge o rimuove risorse localmente;
- implementare il protocollo di **round robin** in caso di fallimento di un download, richiedendo un nuovo Peer al Master finché la lista non si esaurisce.



1.2 Protocollo di Comunicazione

Le interazioni avvengono tramite socket TCP, che consentono lo scambio affidabile di messaggi e dati. Il Master gestisce connessioni concorrenti da parte di più Peer, mentre ciascun Peer può instaurare connessioni sia verso il Master sia verso altri Peer.

L'architettura realizzata può essere definita ibrida perché combina in modo complementare un modello centralizzato e uno distribuito.

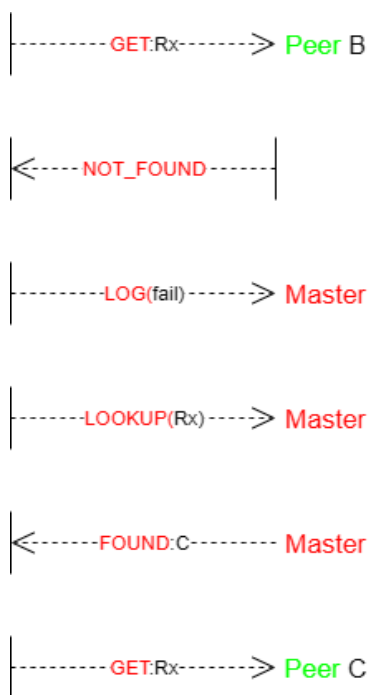
Da un lato, nella fase di gestione delle informazioni, il sistema è centralizzato: il Master rappresenta un punto unico di riferimento che mantiene la tabella globale delle risorse e registra la presenza dei Peer. In questo modo si garantisce un controllo unitario e coerente sullo stato del sistema, evitando duplicazioni e assicurando che tutti i nodi abbiano una visione aggiornata e affidabile delle risorse disponibili.

Dall'altro lato, nella fase di trasferimento delle risorse, l'architettura assume un carattere distribuito, tipico dei sistemi Peer to Peer. Una volta che il Master ha fornito le informazioni necessarie, i Peer stabiliscono connessioni dirette tra loro e gestiscono in autonomia lo scambio dei file. Questo permette di alleggerire il carico sul Master, che non diventa un collo di bottiglia, e di distribuire il traffico di rete, consentendo al sistema di scalare più facilmente con l'aumentare del numero di Peer attivi.

Questa procedura consente, quindi, di sfruttare i vantaggi di entrambi gli approcci: la centralizzazione garantisce chiarezza e affidabilità nella gestione delle informazioni, mentre la distribuzione dei trasferimenti assicura maggiore efficienza e scalabilità.

Inoltre, l'architettura supporta meccanismi di tolleranza ai guasti: nel caso in cui un Peer non sia disponibile o un download fallisca, il Master è in grado di fornire Peer alternativi che possiedono la stessa risorsa, permettendo al sistema di ripetere il tentativo (retry) e garantendo così robustezza e continuità del servizio.

Formato dei Messaggi: Il protocollo di comunicazione prevede messaggi testuali codificati in UTF-8 con formati line-based. I principali tipi di messaggio includono REGISTER, LOOKUP, FOUND, PUBLISH, GET, DATA ed ERROR, che regolano rispettivamente la registrazione, la ricerca delle risorse, l'invio di metadati e la segnalazione di anomalie.



I messaggi sono line-based (terminatore “\n”) e codificati in UTF-8. Esempi di messaggi implementati:

- REGISTER: `REGISTER:<peerId>:<serverPort>` (registrazione Peer con porta server)
- LOOKUP: `LOOKUP:<resourceName>` (richiesta di localizzazione risorsa)
- FOUND: `FOUND:<peerId:ip:port>` (risposta con Peer che possiede la risorsa)
- PUBLISH: `PUBLISH:<resourceName>` (pubblicazione di una risorsa)
- GET: `GET:<resourceName>` (richiesta di download da Peer a Peer)
- DATA: `DATA:<content>` (contenuto della risorsa)
- ERROR: `ERROR:NOT_FOUND` o `ERROR:INVALID_COMMAND` (messaggi di errore)

1.3 Flusso Operativo

Dal punto di vista funzionale, il comportamento del sistema può essere descritto attraverso un flusso tipico articolato in quattro fasi principali:

1. **Registrazione:** quando un Peer si avvia, la prima operazione che compie è connettersi al Master per annunciare la propria presenza nella rete. In questa fase, il Peer invia al Master l'elenco delle risorse locali che intende condividere. Il Master aggiorna quindi la propria tabella interna, mantenendo un quadro globale e costantemente aggiornato delle risorse disponibili e dei nodi che le possiedono. Questa fase è fondamentale per consentire al sistema di essere dinamico; i Peer possono entrare o uscire dalla rete in qualsiasi momento e il Master ne registra in tempo reale lo stato.
Esito: la tabella del Master viene allineata e il Peer diventa immediatamente interrogabile dalle query degli altri nodi. In fase di chiusura, il Peer invia una deregistrazione per rimuovere le proprie entry.
2. **Richiesta:** quando un Peer ha bisogno di accedere a una risorsa che non possiede localmente, invia una richiesta al Master indicando il nome o l'identificativo della risorsa desiderata. Il Master consulta la propria tabella e risponde fornendo l'elenco dei Peer che hanno dichiarato di possedere quella risorsa. In questo modo il Master agisce come mediatore informativo, senza dover trasmettere direttamente i dati, ma facilitando l'incontro tra domanda e offerta di file.
3. **Download:** ricevute le informazioni dal Master, il Peer richiedente seleziona un Peer dalla lista dei possibili fornitori e instaura con esso una connessione diretta. Lo scambio del file avviene in modalità **Peer to Peer**, riducendo il carico sul Master e sfruttando al meglio la distribuzione della rete.

Durante questa fase, il Peer che fornisce la risorsa agisce come un **server locale**, servendo la richiesta del Peer client e permettendo la condivisione dei dati.

Il processo di download è progettato per essere **robusto**:

- I tentativi proseguono finché la lista dei Peer mantenuta dal Master non si esaurisce.
- La selezione dei Peer avviene con politica **round-robin**, così da garantire un bilanciamento del carico e un ordine equo di tentativi.

4. Gestione errori

Può accadere che il download non vada a buon fine, ad esempio perché il Peer selezionato non è più attivo, perché ha rimosso la risorsa dopo averla registrata o perché la connessione viene interrotta. In questi casi, il sistema è progettato per essere tollerante ai guasti: il Peer richiedente torna al Master per ottenere un Peer alternativo. Il Master, a sua volta, aggiorna la tabella delle risorse rimuovendo l'associazione fallita e fornisce un nuovo Peer dalla lista.

Il processo continua secondo una logica di round robin, finché:

- viene trovato un Peer attivo con la risorsa richiesta e il download va a buon fine, oppure
- il Master non ha più Peer disponibili da restituire, nel qual caso viene segnalato che la risorsa non è disponibile sulla rete.

In questo modo il sistema garantisce robustezza e affidabilità, assicurando che, anche in presenza di malfunzionamenti o disconnessioni, le risorse restino accessibili finché esiste almeno un Peer attivo che le possiede.

Gestione delle disconnessioni

La rimozione dei Peer dalla tabella del Master avviene automaticamente quando un Peer si disconnette. Alla chiusura della connessione, il Master procede alla pulizia delle entry associate a quel Peer, mantenendo coerente e aggiornata la tabella globale.

Sincronizzazione e concorrenza

Per garantire la coerenza dei dati, il sistema implementa meccanismi di mutua esclusione lato Master: durante l'aggiornamento della tabella delle risorse, tutte le altre operazioni (query, registrazioni) vengono sospese fino al termine dell'aggiornamento, grazie all'utilizzo di un ReentrantLock.

Lato Peer, invece, il server interno gestisce le richieste in maniera concorrente, creando un thread dedicato per ogni connessione in entrata. Questo consente di servire più download contemporaneamente senza interferenze tra le diverse sessioni.

1.4 Descrizione delle componenti

Il sistema è composto da diverse entità software che collaborano per garantire il corretto funzionamento dell'architettura distribuita. I componenti principali sono il Master, i Peer e le unità logiche interne che ne supportano l'operatività, ciascuno con ruoli e responsabilità specifiche.

Master

Il Master rappresenta il nodo centrale dell'architettura e costituisce il punto di riferimento per l'intero sistema. La sua funzione principale è quella di coordinamento e supervisione: mantiene una tabella globale delle risorse disponibili, aggiornata dinamicamente ad ogni nuova connessione o disconnessione di un Peer, registra i nodi al momento del loro ingresso nella rete e risponde alle richieste di localizzazione dei file. In questo modo assicura che le informazioni sullo stato delle risorse siano sempre coerenti, evitando duplicazioni o dati obsoleti.

A differenza di un server tradizionale, il Master non conserva i file direttamente. Il suo ruolo è informativo e organizzativo: raccoglie e distribuisce metadati sulle risorse, ma delega completamente ai Peer lo scambio effettivo dei contenuti. Questo approccio riduce il rischio che il Master diventi un collo di bottiglia e consente al sistema di mantenere un buon livello di scalabilità.

Dal punto di vista tecnico, la comunicazione con i Peer avviene tramite socket TCP, che garantiscono uno scambio affidabile di messaggi. Ogni nuova connessione è servita da un MasterHandler, ossia un thread dedicato alla gestione della singola comunicazione permettendo al Master di supportare contemporaneamente più Peer senza compromettere le prestazioni complessive. In questo modo, anche in presenza di numerose richieste parallele, il nodo centrale riesce a gestire la concorrenza e a mantenere reattivo il sistema.

Il Master svolge quindi un duplice ruolo: da un lato funge da registro globale delle risorse, dall'altro agisce come arbitro del traffico informativo, garantendo che le interazioni tra Peer avvengano in modo corretto ed efficiente.

Peer

I Peer costituiscono la parte distribuita del sistema e rappresentano i nodi effettivamente coinvolti nello scambio delle risorse. La loro peculiarità è la doppia natura operativa: da un lato agiscono come client nei confronti del Master, comunicando con esso per registrarsi, aggiornare le proprie risorse locali e richiedere informazioni sui file disponibili nella rete; dall'altro lato si comportano come server per gli altri Peer, mettendo a disposizione le proprie risorse e gestendo le richieste in entrata attraverso un server interno dedicato.

Questa duplice funzione consente di mantenere un equilibrio tra centralizzazione delle informazioni (affidata al Master) e decentralizzazione dello scambio dei file (gestita direttamente dai Peer), permettendo all'architettura di essere al tempo stesso affidabile e scalabile.

Ogni Peer dispone di un archivio locale di risorse condivise, costituito dai file che l'utente decide di mettere a disposizione del sistema. La gestione di questo archivio è affidata al componente ResourceManager, che ha il compito di mantenere aggiornato l'elenco delle risorse e fornire supporto operativo al server interno durante le operazioni di condivisione. L'aggiornamento della tabella globale, invece, avviene tramite la logica di **PeerEngine**, che notifica il Master ogni volta che vengono aggiunte o rimosse risorse locali.

Dal punto di vista tecnico, la comunicazione lato client verso il Master si basa su socket TCP e garantisce l'affidabilità delle operazioni di registrazione e richiesta di risorse. Sul lato server, invece, ciascun Peer può ricevere più richieste parallele da parte di altri nodi; per questo motivo la componente di servizio deve essere progettata in modo concorrente, così da assicurare che più trasferimenti possano avvenire simultaneamente senza conflitti.

In sintesi, i Peer non sono semplici terminali passivi, ma veri e propri attori attivi all'interno dell'architettura: contribuiscono sia all'offerta che alla domanda di risorse, bilanciando i carichi e rendendo l'intero sistema più robusto e flessibile.

PeerServer

Elemento chiave dell'architettura è il PeerServer, ovvero la componente interna che consente a ciascun Peer di assumere il ruolo di fornitore di risorse. La sua funzione principale consiste nel ricevere richieste da altri Peer e rispondere inviando i file richiesti, rendendo possibile la condivisione diretta senza passare dal Master.

Dal punto di vista tecnico, il PeerServer è progettato per gestire in modo efficiente scenari di concorrenza: più Peer possono infatti contattare lo stesso nodo simultaneamente per richiedere risorse diverse o addirittura lo stesso file. Ogni connessione viene gestita in un thread dedicato, il che evita che una richiesta possa bloccarne altre. Non sono quindi necessarie sezioni *synchronized* esplicite, perché ogni richiesta lavora su uno stream indipendente; la mutua esclusione è garantita implicitamente dal modello a thread separati.

Il ruolo di questa componente è fondamentale nell'equilibrio complessivo dell'architettura: da un lato consente di scaricare il Master dal traffico legato al trasferimento dei dati, dall'altro rende i Peer attori attivi e autonomi nello scambio delle informazioni. In questo modo, la rete può scalare più facilmente con l'aumentare del numero di nodi, mantenendo prestazioni adeguate anche in presenza di numerose richieste parallele.

ResourceManager

Il ResourceManager è la componente incaricata della gestione delle risorse locali di ciascun Peer, costituendo l'anello di congiunzione tra il file system del nodo e le operazioni di rete. La sua funzione principale è quella di mantenere l'elenco aggiornato dei file che il Peer ha deciso di condividere con la rete, permettendo di aggiungerne di nuovi o verificare la presenza di una risorsa.

Dal punto di vista operativo, il ResourceManager supporta direttamente il PeerServer: quando un altro nodo invia una richiesta di download, è il ResourceManager a fornire al server interno le informazioni necessarie e ad accedere ai file locali per renderli disponibili al trasferimento. Il modulo non notifica direttamente il Master, ma fornisce a PeerEngine le informazioni necessarie affinché quest'ultimo possa comunicare ogni cambiamento allo stato delle risorse.

Il suo ruolo è quindi duplice: da un lato rappresenta una sorta di catalogo locale, che tiene traccia delle risorse disponibili sul Peer; dall'altro funge da interfaccia tra lo spazio di memoria locale e le componenti di rete, garantendo coerenza tra i dati mantenuti localmente e quelli registrati dal Master.

Classi e sotto-componenti

Oltre alle componenti principali già descritte, il sistema si fonda su una serie di classi e sotto-unità logiche che ne assicurano il corretto funzionamento. Un aspetto fondamentale riguarda la gestione della concorrenza: ogni volta che un Peer si connette al Master, quest'ultimo crea un thread dedicato (MasterHandler) incaricato di gestire la singola comunicazione. Questo approccio consente al Master di supportare più connessioni contemporaneamente, senza che l'attività di un Peer blocchi o rallenti quella degli altri.

Anche lato Peer la logica si basa su thread multipli, necessari per garantire la gestione parallela di attività diverse. Un Peer, infatti, deve essere in grado di comunicare con il Master, di mantenere attivo il proprio server interno e, contemporaneamente, di scambiare file con altri Peer. L'uso di thread distinti permette quindi di svolgere queste operazioni in parallelo, evitando colli di bottiglia e assicurando una maggiore fluidità dell'intero sistema.

Dal punto di vista implementativo, il progetto si articola attorno a un insieme di classi fondamentali. MasterMain e MasterCore si occupano rispettivamente dell'avvio e della logica centrale del Master, mentre PeerMain e PeerEngine svolgono lo stesso ruolo per i Peer, offrendo comandi e funzionalità lato utente. La comunicazione tra Peer e Master è gestita dalla classe MasterClient, che funge da interfaccia di collegamento tra le due parti. A queste si affiancano PeerServer, responsabile della gestione delle richieste provenienti da altri Peer, e ResourceManager, che cura la gestione delle risorse locali condivise.

Questa suddivisione in classi e thread rende il sistema modulare, estensibile e soprattutto robusto rispetto alla concorrenza, garantendo che ogni parte dell'architettura possa svolgere il proprio compito senza interferire con le altre.

2. Componenti del progetto (package e file.java)

Il progetto è organizzato in tre package principali: master, peer e shared. Questa struttura consente di separare in modo chiaro le responsabilità tra le varie componenti del sistema, migliorando leggibilità, modularità e manutenibilità del codice.

2.1 Package it.univ.so.master

Contiene le classi che implementano la logica centrale del Master, ossia il nodo di coordinamento dell'architettura.

- MasterMain.java: classe di avvio del Master da linea di comando. Inizializza i componenti principali e avvia il server sulla porta specificata.
- MasterCore.java: implementa la logica del Master, mantenendo la tabella globale delle risorse e fornendo metodi per aggiornarla in base alle registrazioni e disconnessioni dei Peer.
- MasterHandler.java: gestisce le singole connessioni dei Peer al Master. Ogni connessione è servita da un thread dedicato, che traduce i messaggi ricevuti in operazioni sul MasterCore.

2.2 Package it.univ.so.peer

Contiene le classi che definiscono il comportamento dei Peer, i quali assumono sia il ruolo di client verso il Master, sia quello di server verso gli altri Peer.

- PeerMain.java: classe di avvio del Peer da linea di comando. Si occupa dell'inizializzazione di PeerEngine, PeerServer e ResourceManager.
- PeerEngine.java: rappresenta il cuore logico del Peer. Implementa il ciclo REPL (Read-Eval-Print Loop) con i comandi listdata, add, download, quit. Gestisce le interazioni con il Master tramite MasterClient e coordina le risorse locali con ResourceManager.
- PeerServer.java: server interno che consente al Peer di rispondere a richieste di download da parte di altri nodi. Ogni richiesta è gestita in un thread dedicato, garantendo la concorrenza tra più operazioni.
- MasterClient.java: modulo di interfaccia tra Peer e Master. Si occupa di inviare messaggi di registrazione, lookup e log, e ricevere le risposte necessarie.
- ResourceManager.java: gestisce l'archivio locale delle risorse condivise dal Peer, fornendo i metodi per aggiungere nuove risorse, verificarne la disponibilità e restituirne il contenuto. Collabora con PeerEngine e PeerServer.

2.3 Package it.univ.so.shared

Contiene le classi di supporto comuni a Master e Peer, che rappresentano le entità logiche di base della rete.

- Resource.java: modello che rappresenta una risorsa condivisa, con attributi come nome e contenuto.
- NetworkRecord.java: struttura dati utilizzata per memorizzare le informazioni relative a un Peer (id, indirizzo IP, porta e risorse possedute).

2.4 Stack tecnologico

Il progetto è stato sviluppato in Java 17 (LTS) ed è organizzato tramite Maven, così da garantire un ambiente di lavoro ordinato, modulare e facilmente portabile su diverse macchine. L'uso di Maven consente inoltre di automatizzare il processo di compilazione ed esecuzione, evitando configurazioni manuali e assicurando una gestione coerente delle dipendenze. Per il versionamento del codice e la collaborazione di gruppo è stato utilizzato Git, con repository ospitato su piattaforma GitHub/GitLab in modalità privata, come richiesto dalle specifiche del corso. L'applicazione finale viene distribuita sotto forma di file eseguibile .jar, che contiene tutti i file compilati e permette di avviare facilmente sia il Master che i Peer tramite linea di comando.

3. Meccanismi di concorrenza e sincronizzazione

In questa sezione vengono descritti i principali problemi di concorrenza emersi durante lo sviluppo del sistema distribuito e le soluzioni adottate per garantire correttezza ed efficienza nell'esecuzione.

3.1 Problemi riscontrati

Durante l'implementazione del sistema distribuito, sono stati identificati e risolti diversi problemi di concorrenza:

- Concorrenza sul Master: il Master deve gestire simultaneamente più connessioni da parte di Peer diversi, ognuna delle quali può richiedere operazioni di lettura e scrittura sulla tabella delle risorse. Senza meccanismi di sincronizzazione, si potrebbero verificare race condition durante l'aggiornamento della tabella.
- Richieste parallele sui Peer: i Peer devono essere in grado di gestire più richieste di download simultaneamente da parte di altri Peer, mantenendo l'integrità dei dati durante il trasferimento.
- Gestione risorse inesistenti: durante le operazioni di download, un Peer potrebbe richiedere una risorsa che nel frattempo è stata rimossa o il Peer che la possedeva si è disconnesso.
- Prevenzione deadlock: è necessario evitare situazioni in cui due o più thread si bloccano reciprocamente in attesa di risorse.

3.2 Soluzioni adottate

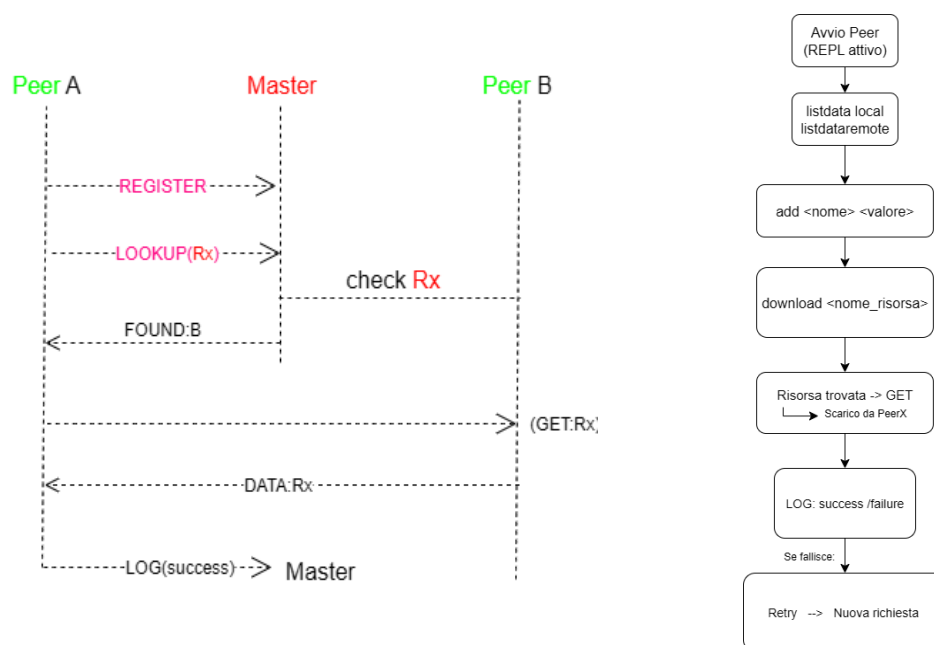
Per affrontare i problemi descritti sono stati implementati i seguenti meccanismi:

- PeerServer.java: gestione concorrente delle connessioni tramite thread dedicati a ciascun client remoto. Questo modello evita interferenze tra richieste parallele e riduce il rischio di colli di bottiglia.
- MasterCore.java: utilizzo di ReentrantLock per proteggere l'accesso concorrente alla tabella delle risorse, garantendo atomicità nelle operazioni di aggiornamento ed evitando race condition.
- Download sicuri: implementazione di un meccanismo basato su round robin: in caso di fallimento di un download, il Peer torna al Master per ottenere un nuovo Peer. Il Master aggiorna la tabella rimuovendo l'entry fallita e restituisce un'alternativa, finché la lista non si esaurisce.
- Gestione log: la consistenza dei log è garantita dal fatto che ogni richiesta di logging viene serializzata dal Master tramite i thread MasterHandler. In questo modo le registrazioni non si sovrappongono e l'ordine degli eventi resta corretto.

4. Funzionalità e utilizzo

4.1 Comandi lato Peer (CLI - Command Line Interface)

Il Peer offre un'interfaccia a riga di comando (CLI) che consente di gestire le risorse locali e interagire con il Master.



Comandi disponibili:

- listdata local

- Descrizione: mostra la lista delle risorse disponibili localmente sul Peer, cioè i file che sono stati aggiunti tramite il comando “add”.
- Esempio con risorse disponibili:

```
> listdata local  
- file1.txt
```

- Se non ci sono risorse locali:

```
> listdata local  
Nessuna risorsa locale.  
>
```

- listdata remote

- Descrizione: richiede al Master la lista delle risorse disponibili nella rete, indicando per ogni risorsa i Peer che la possiedono (in formato round-robin).
- Esempio con più risorse replicate su più Peer:

```
> listdata remote  
Risorse disponibili nella rete:  
- : (nessun Peer)  
> |
```

- Se nessuna risorsa è disponibile nella rete:

```
Connesso al Master. In attesa di comandi...  
> listdata remote  
Risorse disponibili nella rete:  
- file1.txt : Peer-1228,Peer-9678  
- file3.txt : Peer-3816  
- file2.txt : Peer-1228,Peer-9678
```

- add <nome_risorsa> <contenuto>

- Descrizione: aggiunge una nuova risorsa locale e la pubblica nella rete
- Esempio di aggiunta:

```
> add file1.txt "contenuto file1"  
Risorsa aggiunta: file1.txt  
> add file2.txt "contenuto file2"  
Risorsa aggiunta: file2.txt
```

- Se il comando è malformato:

```
> add file3.txt  
Uso: add <nome_risorsa> <contenuto>  
> |
```

- `download <nome_risorsa>`
 - Descrizione: scarica una risorsa dalla rete. Il Peer chiede al Master quali Peers la possiedono e tenta il download diretto da uno di essi.
 - La logica implementata prevede:
 1. Evitare di scaricare da sé stesso.
 2. Provare più Peer in sequenza (round-robin).
 3. Registrare un log di successo o fallimento.
 - Esempio di download riuscito:

```
> download file1.txt
Tentativo 1: download di file1.txt da Peer-1228
Download completato da Peer-1228.
```

- Esempio di risorsa inesistente:

```
> download file5.txt
Tentativo fallito per il download di file5.txt per il richiedente: Peer-5080.
Risorsa inesistente.
> download file2.txt
Tentativo fallito per il download di file2.txt per il richiedente: Peer-5080.
Risorsa inesistente.
```

- Esempio download risorsa già disponibile localmente

```
> download file.txt
Risorsa file.txt già disponibile localmente.
> |
```

- `quit`
 - Descrizione: chiude la connessione del Peer, informando il Master che il nodo non è più disponibile. In questo modo il Master rimuove automaticamente le risorse del Peer dalla propria tabella.

```
> quit
Chiusura del Peer...
|
```

4.2 Comandi lato Master (console)

Il Master supporta i seguenti comandi dalla console CLI (Command Line Interface). Questi comandi sono utili per monitorare lo stato della rete e gestire la chiusura ordinata del server centrale.

- `log`
 - Descrizione: il log delle operazioni di download (sia di successo che di insuccesso).
 - Esempio log di un download riuscito:

```
Log dei download:
- 14:15 file1.txt da: Peer-4911 a: Peer-6669 SUCCESS
```

- Se non ci sono download registrati:

```
log
Nessun download registrato.
```

- quit
 - Descrizione: arresta il Master e chiude la rete
 - Esempio:

```
quit
Avvio chiusura del Master...
Chiusura ordinata del Master...
Master terminato.
```

- add
 - Descrizione: Il Master non ha un comando diretto per pubblicare, ma riceve notifiche automatiche dai Peer tramite il comando add. Ogni volta che un Peer pubblica una risorsa, il Master registra l'evento.
 - Esempio:

```
+ Peer-4911 pubblica risorsa "file1.txt"
+ Peer-4680 pubblica risorsa "file1.txt"
+ Peer-4911 pubblica risorsa "file2.txt"
+ Peer-4680 pubblica risorsa "file2.txt"
```

- download
 - Descrizione: Il Master non ha un comando diretto per scaricare risorse, ma riceve le notifiche automatiche dai Peer tramite il comando download. Ogni volta che un Peer scarica una risorsa, il Master registra l'evento.

```
LOG: 13:25 file1.txt da: Peer-1228 a: Peer-5080 SUCCESS
LOG: 13:25 file5.txt da: NO_SOURCE a: Peer-5080 FAILED
```

```
LOG: 13:26 file2.txt da: NO_SOURCE a: Peer-5080 FAILED
Risorsa "file3.txt" rimossa dal Master.
```

5. Compilazione ed esecuzione

Di seguito vengono fornite le istruzioni pratiche per compilare ed eseguire il sistema, passo dopo passo.

5.1 Uso di Maven

Il progetto utilizza Maven per la gestione delle dipendenze e la compilazione.

Comando di compilazione

Dalla cartella principale del progetto eseguire:

```
mvn clean package
```

Risultato

Il comando:

- clean: elimina eventuali build precedenti,
- package: compila i sorgenti, esegue i test e genera il file .jar.

Posizione del file .jar generato

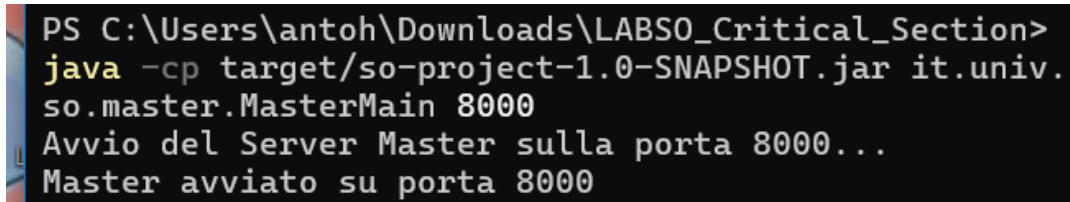
Al termine, Maven genera l'eseguibile in:

target/so-project-1.0-SNAPSHOT.jar

5.2 Avvio manuale da terminale

Per avviare il Master sulla porta desiderata (es. 5000):

```
java -cp target/so-project-1.0-SNAPSHOT.jar it.univ.so.master.MasterMain 8000
```

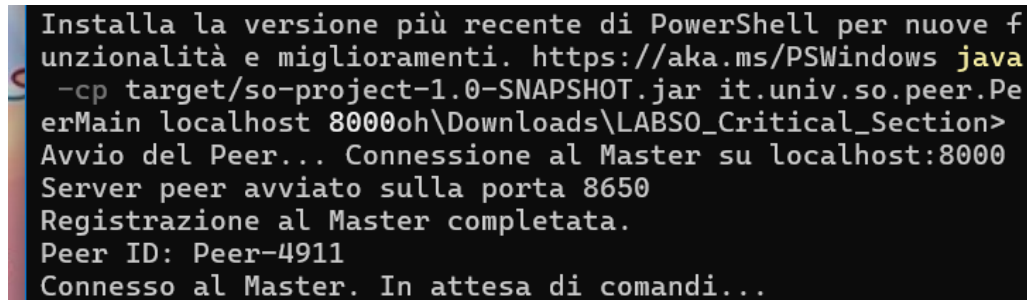


```
PS C:\Users\antoh\Downloads\LABSO_Critical_Section>
java -cp target/so-project-1.0-SNAPSHOT.jar it.univ.
so.master.MasterMain 8000
Avvio del Server Master sulla porta 8000...
Master avviato su porta 8000
```

5.3 Avvio di un Peer

Per avviare un Peer e collegarlo al Master (IP 127.0.0.1, porta 5000):

```
java -cp target/so-project-1.0-SNAPSHOT.jar it.univ.so.peer.PeerMain 127.0.0.1 8000
```



```
Installa la versione più recente di PowerShell per nuove f
unzionalità e miglioramenti. https://aka.ms/PSWindows java
-cp target/so-project-1.0-SNAPSHOT.jar it.univ.so.peer.Pe
erMain localhost 8000oh\Downloads\LABSO_Critical_Section>
Avvio del Peer... Connessione al Master su localhost:8000
Server peer avviato sulla porta 8650
Registrazione al Master completata.
Peer ID: Peer-4911
Connesso al Master. In attesa di comandi...
```

6. Conclusione e riflessioni finali

Il progetto ha permesso di sviluppare un sistema distribuito basato su architettura Master–Peer, in grado di gestire la condivisione e il download di risorse tra nodi.

Durante le fasi di implementazione e test sono stati raggiunti i seguenti risultati principali:

- Funzionamento completo del ciclo di vita delle risorse: dalla pubblicazione (add) alla ricerca (listdata remote), fino al download e alla registrazione dei log sul Master.
- Gestione multi–Peer: il Master è in grado di gestire più connessioni concorrenti in maniera stabile grazie all’uso di thread dedicati (MasterHandler) e meccanismi di sincronizzazione (ReentrantLock).
- Robustezza nelle operazioni: grazie alla logica di round robin sui download, il sistema resta affidabile anche in presenza di Peer non raggiungibili o che hanno rimosso risorse, provando fornitori alternativi fino all’esaurimento della lista.
- Configurabilità: tramite il file config.properties è possibile modificare facilmente i parametri principali (indirizzo e porta del Master, porta del Peer, timeout di connessione), senza dover ricompilare il codice.
- Tracciabilità: i log dettagliati gestiti dal Master consentono di monitorare tutte le operazioni di download effettuate nella rete, distinguendo successi e fallimenti.

In sintesi, il progetto ha permesso di affrontare problematiche reali legate a concorrenza, sincronizzazione e comunicazione distribuita, fornendo un sistema semplice ma estendibile.

7. Suddivisione del lavoro nel gruppo

La realizzazione del progetto è stata condotta attraverso un approccio collaborativo basato principalmente sul peer programming e sul confronto costante tra i membri del gruppo. Piuttosto che suddividere rigidamente i compiti tra i partecipanti, abbiamo scelto di lavorare insieme sulle diverse componenti, condividendo in tempo reale idee, soluzioni e problematiche incontrate durante lo sviluppo. Questo metodo ha permesso di accrescere la comprensione collettiva del progetto e di ridurre i rischi legati a fraintendimenti o implementazioni incoerenti.

In pratica, durante le sessioni di lavoro, uno dei membri assumeva il ruolo di driver, occupandosi della scrittura effettiva del codice, mentre gli altri svolgevano il ruolo di navigator, supervisionando le scelte progettuali, verificando la correttezza logica e suggerendo miglioramenti o correzioni. I ruoli sono stati

ruotati frequentemente, in modo da garantire che tutti acquisissero familiarità sia con gli aspetti implementativi sia con quelli concettuali del progetto.

Abbiamo dedicato una parte significativa del tempo alla revisione collettiva del codice, effettuata subito dopo ogni implementazione rilevante. Questo ci ha consentito di intercettare rapidamente errori o potenziali criticità legate alla concorrenza e alla gestione delle risorse, oltre a garantire uniformità nello stile di programmazione. La documentazione è stata redatta anch'essa in maniera condivisa: ogni sezione è stata discussa in gruppo prima della stesura finale, così da riflettere al meglio l'esperienza comune e non la prospettiva di un singolo.

Infine, l'uso congiunto di strumenti di collaborazione (GitHub per il versionamento, IDE condivisi, e canali di comunicazione costante) ha rafforzato ulteriormente l'efficacia del metodo. Questo approccio ha permesso non solo di portare a termine il progetto in maniera coerente e robusta, ma anche di sviluppare competenze trasversali fondamentali, come la capacità di lavorare in team, il problem solving collaborativo e la condivisione di responsabilità.